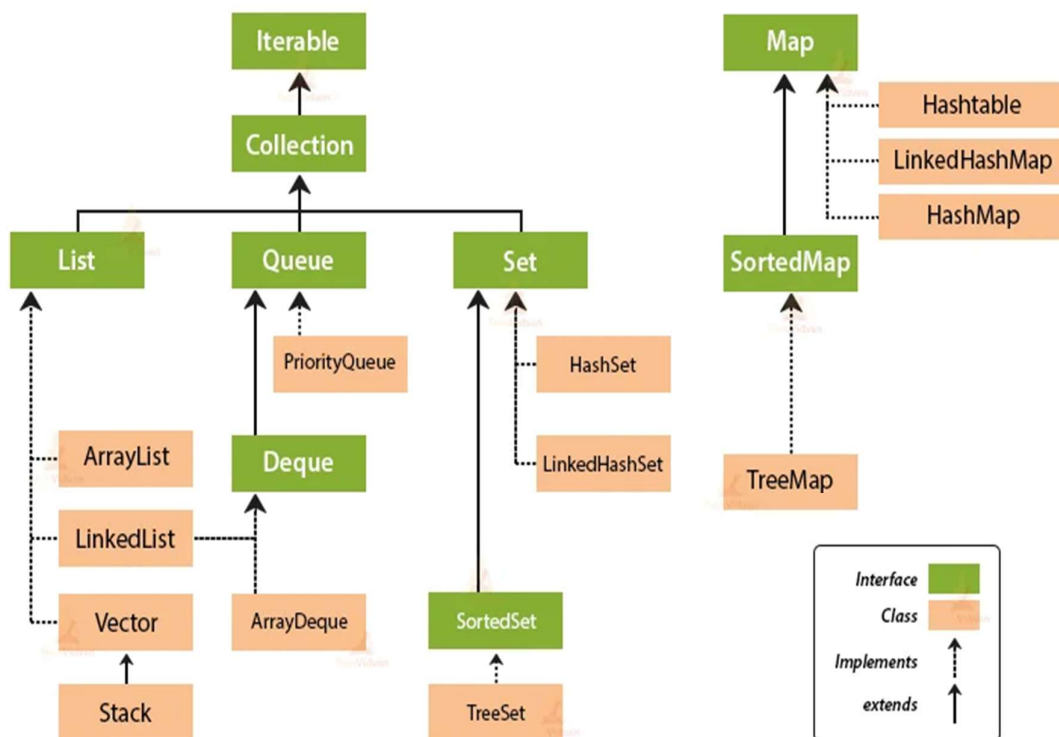


Collection Framework

What is Collection Framework :-

- The Collection Framework is a set of **classes** and **interfaces** that help **store and manipulate groups of objects** efficiently.
- It provides **data structures** like (**lists, sets, maps, queues**) and **algorithms** (like **sorting, searching**, etc.) that make it easier to handle large amounts of data.
- The collection framework is part of Java's **java.util** package.

Collection Framework Hierarchy in Java



The Collection Framework :-

- ✓ **Interfaces** (List, Set, Map, Queue)
- ✓ **Classes** (ArrayList, HashSet, HashMap, etc.)
- ✓ **Algorithms** (sorting, searching, etc.)

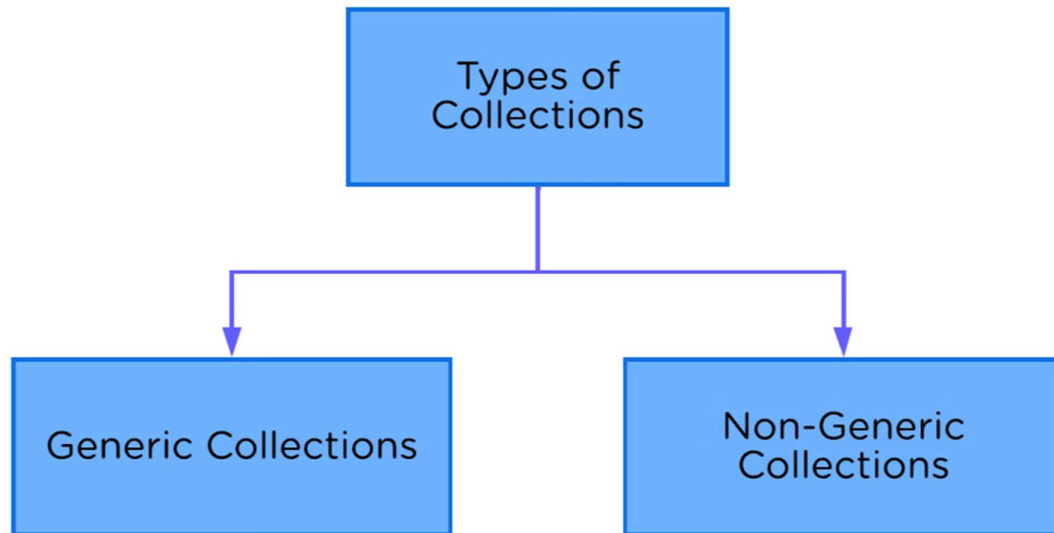
Why we need Collection Framework in java :-

- We need the Collection because it makes it **easier, faster, and more efficient to handle and organize large amounts of data.**
- Dynamic Size
- Data Manipulation Made Easy
- Unified Architecture
- Better Performance
- Reusability and Maintenance
- Type Safety

Advantages of Collection Framework :-

- **Reusability** – Ready-to-use data structures.
- **Performance** – Optimized implementations.
- **Flexibility** – Can switch between different implementations easily.
- **Reduced code** – No need to write complex data structures manually.

✚ Types of Collection :-



✚ Generic Collection :-

- It can store **same type of data**.
- It can including **Homogeneous** Object.
- **Advantages:** No type casting, fewer runtime errors, better performance.

✚ Non - Generic Collection :-

- It can store **any type of object**.
- It can including **Heterogeneous** Object.
- **Disadvantages:** Type casting is required and may cause runtime errors.

Difference Between Generic and Non - Generic Collection :-

	Base	Generic Collections	Non-Generic Collections
 Syntax		Use parameterized types (e.g., ArrayList<String>)	Use raw types (e.g., ArrayList) without type specification
 Type-safety		Strongly typed, ensures data integrity	Not strongly typed, potential data integrity issues
 Typecasting		Not required when retrieving elements	Required when retrieving elements
 Compile-Time Checking		Detects type mismatches at compile-time	No compile-time type checking
 Reusability		Can be reused for different data types	Less reusable due to lack of type specificity
 Code Readability		Enhances code clarity	May lead to confusion due to lack of type information
 Performance		Potentially better due to optimized type handling	May involve runtime type checks and casting

Syntax of collection :-

List names = new ArrayList();

OR

ArrayList Student = new ArrayList();

Methods of Collection interface in java :-

java.util.Collection

- **boolean add(Object obj)**
- Adds the **obj** to the invoking collection.
- **boolean addAll(Collection c)**
- Adds all the elements of **c** to the invoking collection.
- **boolean remove(Object obj)**
- Removes the **obj** from the invoking collection.
- **boolean removeAll(Collection c)**
- Removes all the elements of **c** from the invoking collection.
- **boolean retainAll(Collection c)**
- Removes all the elements from the invoking collection except elements in **c**.
- **void clear()**
- Removes all elements from the invoking collection.
- **boolean contains(Object obj)**
- Finds the **obj** to the invoking collection.
- **boolean containsAll(Collection c)**
- Finds all elements of **c** in the invoking collection.
- **boolean equals(Object obj)**
- Returns true if the **obj** and the invoking collection are equal, else returns false.
- **boolean isEmpty()**
- Returns true if the invoking collection is empty, else returns false.
- **int size()**
- Returns total number of elements in the invoking collection.
- **int hashCode()**
- Returns hash code for the invoking collection.
- **Object[] toArray()**
- Returns an array that contains all elements of the invoking collection.
- **Object[] toArray(Object obj)**
- Returns an array that contains only those elements whose type matches the array type.
- **Iterator iterator()**
- Returns an iterator for the invoking collection.

Implementation of Collection :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Collection {
7
8     public static void main(String[] args) {
9
10         List names = new ArrayList();
11         names.add("Nikhil");
12         names.add(23);
13         names.add(74997658881);
14         names.add("Software Developer");
15
16         System.out.println(names);
17     }
18 }
```

Output :-

[Nikhil, 23, 7499765888, Software Developer]